ASSIGNMENT 8.5

Hemavathi N
2303A51965
Batch 24
AIAC

Task Description #1 (Username Validator – Apply AI in Authentication Context)
• Task: Use AI to generate at least 3 assert test cases for a function is_valid_username(username) and then implement the function using Test-Driven Development principles.
CODE:

```
def is_valid_username(username):
    if len(username) < 5 or len(username) > 15:
        return False
    if not username[0].isalnum():
        return False
    for char in username:
        if not char.isalnum() and char != '_':
            return False
    return True
# Test cases for the is_valid_username function
assert is_valid_username("user123") == True, "Test case 1 failed"
assert is_valid_username("1user") == True, "Test case 2 failed"
assert is_valid_username("user_name") == True, "Test case 3 failed"
assert is_valid_username("us") == False, "Test case 4 failed"
print("Username validation logic successfully passing all AI-generated test cases.")
```

OUTPUT:

```
Username validation logic successfully passing all AI-generated test cases.
```

Task Description #2 (Even–Odd & Type Classification – Apply AI for Robust Input Handling)
• Task: Use AI to generate at least 3 assert test cases for a
function classify_value(x) and implement it using conditional logic and loops.
CODE:

```
def classify_value(x):
    if x<0:
        return "Negative"
    if x%2==0:
        return "Even"
    elif x%2!=0:
        return "Odd"
    if isinstance(x, str):
        return "Invalid Input"
    #examples for assert testcases
    assert classify_value(0)== "Even", "Test case 1 failed"
    assert classify_value(7)== "Odd", "Test case 2 failed"
    assert classify_value("abc")== "Invalid Input", "Test case 3 failed"
    print("Function correctly classifying values and passing all test cases.")
```

Task Description #3 (Palindrome Checker – Apply AI forString Normalization)
• Task: Use AI to generate at least 3 assert test cases for a function is_palindrome(text) and implement the function.
CODE:

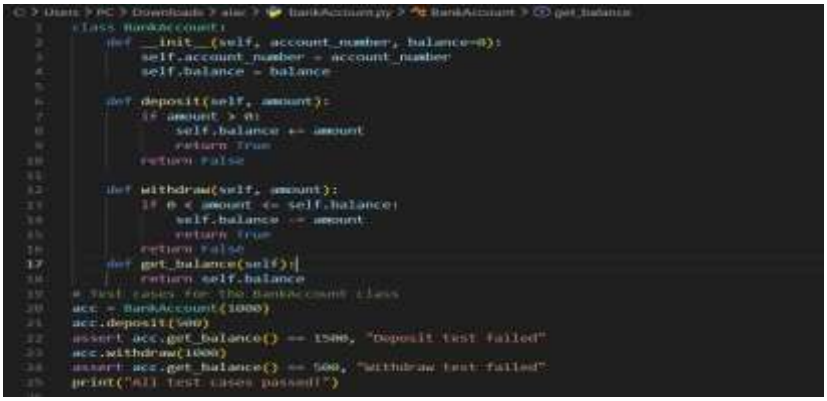
```
def is_palindrome(text):
    cleaned_text = ''.join(char.lower() for char in text if char.isalnum())
    return cleaned_text == cleaned_text[::-1]
# Test cases for the is_palindrome function
assert is_palindrome("A man, a plan, a canal panama") == True, "Test case 1 failed"
assert is_palindrome("Hello") == False, "Test case 2 failed"
assert is_palindrome("Hi") == False, "Test case 3 failed"
print("Function correctly identifying palindromes and passing all AI-generated tests")
```

OUTPUT:

```
Function correctly identifying palindromes and passing all AI-generated tests
```

Task Description #4 (BankAccount Class – Apply AI for Object-Oriented Test-Driven Development)
• Task: Ask AI to generate at least 3 assert-based test cases for a BankAccount class and then implement the class.

CODE:



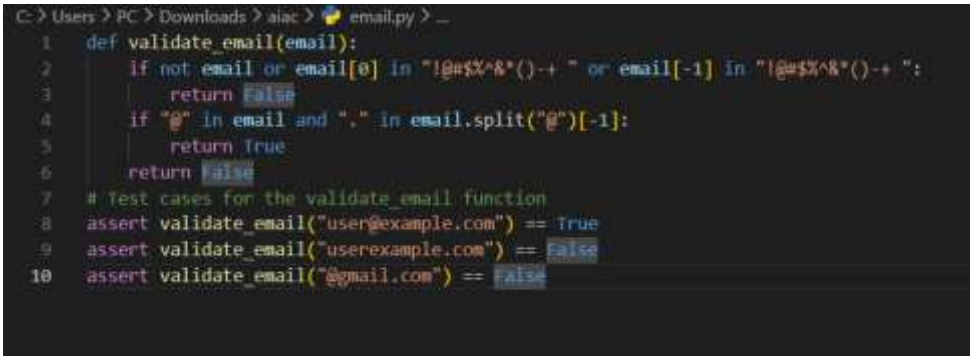
```
class BankAccount:
    def __init__(self, account_number, balance=0):
        self.account_number = account_number
        self.balance = balance

    def deposit(self, amount):
        if amount > 0:
            self.balance += amount
            return True
        return False

    def withdraw(self, amount):
        if 0 < amount <= self.balance:
            self.balance -= amount
            return True
        return False
    def get_balance(self):
        return self.balance
# Test cases for the BankAccount class
acc = BankAccount(1000)
acc.deposit(500)
assert acc.get_balance() == 1500, "Deposit test failed"
acc.withdraw(1000)
assert acc.get_balance() == 500, "Withdraw test failed"
print("All test cases passed!")
```

Task Description #5 (Email ID Validation – Apply AI for Data Validation)
• Task: Use AI to generate at least 3 assert test cases for a function validate_email(email) and implement the function.

CODE:



```
def validate_email(email):
    if not email or email[0] in "!@#$%^&*()-+ " or email[-1] in "!@#$%^&*()-+ ":
        return False
    if "@" in email and "." in email.split("@")[-1]:
        return True
    return False
# Test cases for the validate_email function
assert validate_email("user@example.com") == True
assert validate_email("userexample.com") == False
assert validate_email("@gmail.com") == False
```

OUTPUT:

```
PS C:\Users\PC> & C:/Users/PC/AppData/Local/Programs/Python/Python314/python.exe c:/Users/PC/Downloads/aiac/email.py
```